Vex is a compact, statically typed, procedural (at least in the first version), toy programming language. I made it to experiment with compiler front-ends and IR optimization.

Vex syntax was made, somewhat carefully, somewhat for fun, for the following goals:

- (i) Make compilation and IR optimization simpler (designed to lower cleanly into SSA-style IR)
- (ii) Loosely resemble C-syntax, with some Python-like syntax sugar
- (iii) Be an 'ideal' language for sport programming (at least, according to the author's biases)

The first version will focus on (i), since the main goal of this project is for me to play around with IR optimization. In later versions, features for (ii) and (iii) can be factored in.

An example program in Vex, which (rather inefficiently) prints the first 15 Fibonacci numbers:

```
fun fib(i32 n) -> i32 {
    if n <= 1:
        return n;
    else: {
        i32 a = fib(n - 1), b = fib(n - 2);
        return a + b;
    }
}

for i in [1..15]:
    println(fib(i));
```

0. Core ideas

Declarations are explicit, statements end in semicolons, and conditions end with a colon before the block opens. Blocks `{...}` are multi-line codes that act as a single-line with their own scope. Indentation doesn't matter. There are no structs or classes, at least in the first version. To exit the running program, a keyword called `exit(return code)` is defined.

1. Pre-defined Variable Types

- `i32` type : Basic integer type, 32-bits
- `bool` type: Basic Boolean type, stores true/false values
- `nil` type: Basic empty or 'null' type
- Standard stack-allocated arrays, which store a singular type.
  Future versions may implement: `i64`, `float`, `char`, `string` *
  More extreme: A built-in modular integer type `m32` and `m64`, optimized at compile time.

* - **Note:** String literals are supported, however, they can only be used with a `print` function as of now. See "Standard I/O" for more details.

2. Basic Operators

- Unary Minus (`-x`)
- Arithmetic (`+`, `-`, `*`, `/`, `%`)
- Comparators (`>`, `>=`, `<`, `<=`, `==`, `!=`)

- Logical (`&&` , `||` , `!`)
  Future versions may implement: `//` , `++` , `--` , `+=` , `-=` , etc., Bit-wise operators
  More extreme: Modular operation support for `m32` and `m64` .

Example. Variables and arithmetic

```
i32 a = 5;
i32 b = 8;
i32 c = a * 2 + b / 4;
println(c);
```

3. Branches

Conditional branches are supported by `if` , `elif` , and `else` statements, similar to Python.

Example. Branching

```
if ... : {
    # do something
}
elif ... : {
    # otherwise if
}
else: {
    # else do this
}
```

Here, `elif` is sugar for nested if/else and lowers away during parsing.

4. Functions

- Functions are defined with the `fun` keyword
- Fixed number of arguments, all types, including the return type, need to known at compile time

```
fun sum_2(i32 a, i32 b) -> i32 {
    i32 total = a + b;
    return total;
}
```

5. Arrays

- Arrays are homogeneous and fixed-size in the first version.
- Array access is explicit.
- They can be passed to functions by reference.
- They are not aware of their own size

```
fun sum_4(i32 arr[], i32 n) -> i32 {
    if n < 4: exit(1);
    i32 total = arr[0] + arr[1];
    total = total + arr[2];
    total = total + arr[3];
```

```
    return total;
}

i32 arr2[20];
arr2[0] = 1; arr2[1] = 2;
arr2[2] = 3; arr2[3] = 5;
print(sum_4(arr2, 20));
```

6. Loops

- While loops:

```
i32 i = 1;
while i <= 15: {
    println(fib(i));
    i = i + 1;
}
```

- Range-based for loops: (inclusive of range endpoints)

```
i32 i;
for i in [1..15]:
    println(fib(i));
```

7. Standard I/O

Built-in to the language. Scan using

```
read(var1, var2, ...);
```

Output using

```
print(var1, var2, ...); # space-separated
println(var1, var2, ...); # space-separated, followed by a newline
```

Since the basic version of Vex doesn't have String support, there exist string literals whose sole purpose and only use is with the `print` and `println` commands, like so:

```
i32 a, b;
read(a, b);
i32 c = a + b;
println("Your sum, ", a, " + ", b, " equals ", c);
```